



Department of Information Technology, State Polytechnic of Malang
Business Information Systems Study Program
Object-Oriented Programming Course(Practical Course)
Quiz 1

Quiz Objectives

After completing this quiz, students are expected to be able to:

1. Demonstrate an understanding of basic Object-Oriented Programming (OOP) concepts.
2. Implement classes and objects using Java.
3. Apply encapsulation using private attributes, constructors, getters, and setters.
4. Identify and implement appropriate relationships between classes.
5. Develop a simple Java program based on a given case study using OOP principles.

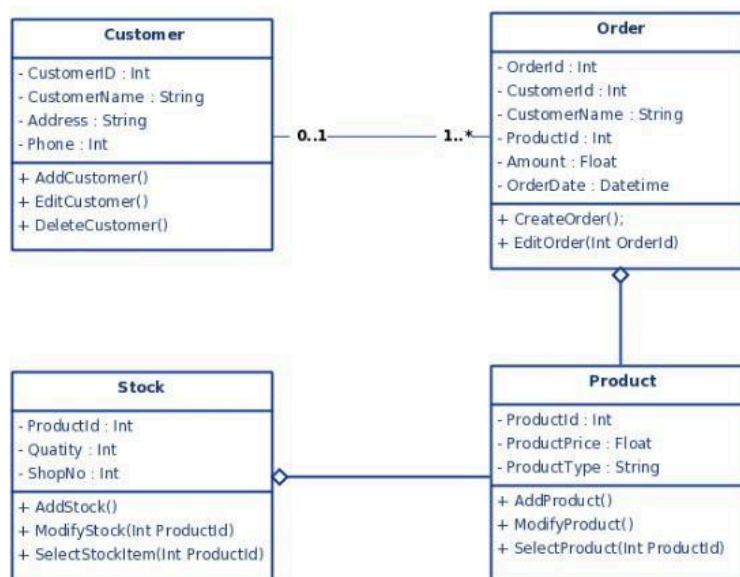
Quiz Rules and Requirements

1. This quiz must be completed individually.
2. Submit one PDF file containing:
 - o The complete program code.
 - o Screenshots showing the program output.
3. Copying or using code from classmates or AI tools is strictly prohibited.
4. Complete the quiz using your own skills and understanding. Be confident, trust your abilities, and do your best.

Quiz:

1. Examine the following class diagram. Correct the class diagram according to proper UML notation and justify the corrections you make. After revising the diagram, explain the relationships between the classes and the UML notations used. Finally, implement the corrected class diagram as a Java program.

Class Diagram for Order Processing System



2. An automotive workshop called “**Bengkel Maju**” wants to develop a simple system to manage customer vehicle data for vehicles brought in for repair. The Java-based system should be able to record vehicle information (cars and motorcycles), display service information, and calculate the estimated cost based on the type of vehicle and type of service. The system should implement object-oriented programming principles:

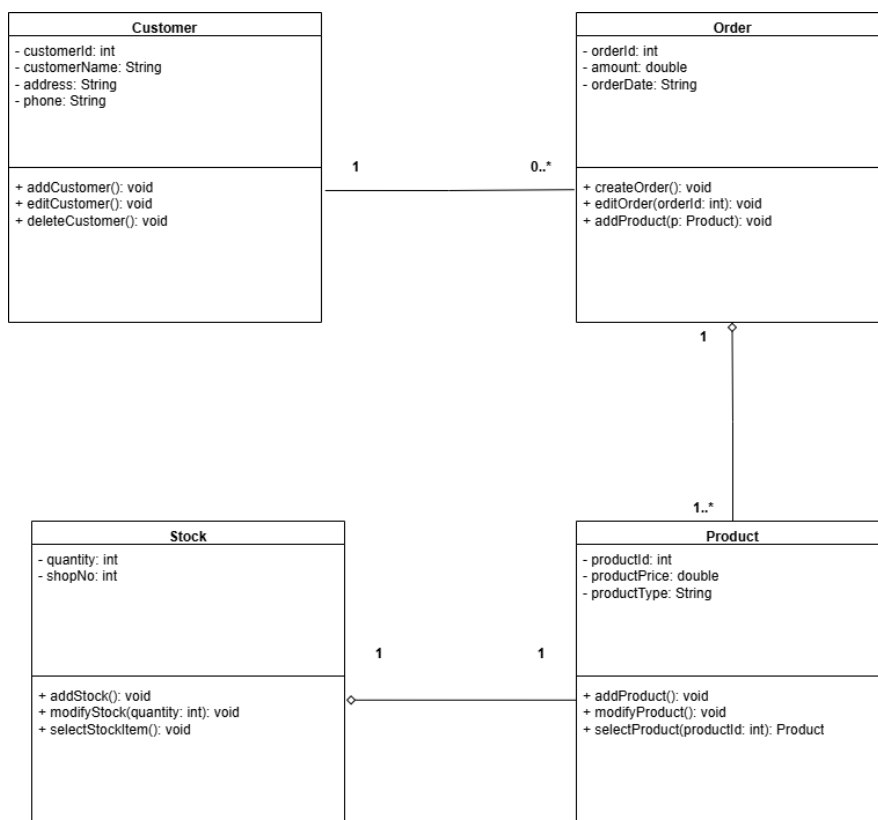
1. *Customer class* with the attributes `name` and `phoneNumber`.
2. *Vehicle class* with the attributes `plateNumber`, `brand`, `model`, and `vehicleType`.
3. *Service class* with the attributes `serviceName` and `servicePrice`.
4. Add one *Employee class* with appropriate attributes and methods based on the system requirements.
5. Apply *encapsulation* to all attributes using *private* attributes, constructors, getters, and setters.
6. Establish relationships between the classes so that one customer can have multiple vehicles, while each vehicle belongs to only one customer. Define appropriate relationships involving the *Service* and *Employee* classes.
7. Create a *Main* program to instantiate objects and display all service information.
8. Calculate the total service cost according to the following rules:
 - **Car** → service cost + Rp50,000 (service fee)
 - **Motorcycle** → service cost + Rp20,000 (service fee)

Output:

Display the customer’s name, vehicle information (including some example of 2 cars and 2 motorcycles), selected service, and total estimated cost.

Answer

1.



a. Solid line for Customer - Order association

The original diagram used a dashed line between customer and order. In UML, dashed lines represent a temporary dependency or interface realization. Because customer and order have a persistent structural relationship, it was replaced with a solid association line.

b. Fixed inverted multiplicity

The initial diagram showed 0..1 on the customer end and 1..* on the order end, which implied that an order could exist without an owner. This was swapped to 1 on customer and 0..* on order: every order must belong to exactly one customer, while a customer can exist before placing any orders.

c. Removed database-style foreign keys

Fields like CustomerId, CustomerName, and ProductId inside Order, as well as ProductId inside stock, were stripped out. In OOP class modeling, relationships are represented by associations and direct object references rather than relational table foreign keys.

d. Corrected aggregation notations and missing bounds

Hollow diamond symbols (◇) were placed at the container ends (Order and Stock) to properly signify shared aggregation. The missing multiplicity numbers were also filled in: 1 order aggregates 1..* products, and 1 stock item corresponds to 1 product.

e. Standardized method signatures and return types

The original operations lacked return types and used non-standard parameter styling. They were updated to match the standard UML format `methodName(param: Type): ReturnType`, such as `editOrder(orderId: int): void` and `selectProduct(productId: int): Product`.

f. Cleaned up typos and data types

The typo `Quatity : Int` in the stock class was corrected to `quantity: int`. PascalCase and non-standard types like `int`, `float`, and `datetime` were converted to standard representations: `int`, `double`, and `String`.

RESULT:

```
>>> 1. Inisialisasi dan Penambahan Pelanggan <<<
[Customer] Data pelanggan 'Agam Darmawan' (ID: 101) berhasil ditambahkan.

>>> 2. Inisialisasi Produk dan Pengaturan Stok <<<
[Product] Produk ID 1001 (Laptop Gaming) berhasil didaftarkan ke sistem.
[Product] Produk ID 1002 (Mechanical Keyboard) berhasil didaftarkan ke sistem.
[Stock] Stok awal sebanyak 15 unit ditambahkan pada Toko #1.
[Stock] Stok awal sebanyak 30 unit ditambahkan pada Toko #1.
[Stock] Stok produk ID 1001 pada Toko #1 diubah menjadi: 12 unit.
[Stock] Toko #1 | Produk ID: 1001 (Laptop Gaming) | Sisa Stok: 12
[Stock] Toko #1 | Produk ID: 1002 (Mechanical Keyboard) | Sisa Stok: 30

>>> 3. Pembuatan Order dan Agregasi Produk <<<
[Order] Pesanan ORD-501 dibuat pada tanggal 2026-09-22 untuk Agam Darmawan.
[Order] Produk ID 1001 (Laptop Gaming) ditambahkan. Harga: Rp8500000.0
[Order] Produk ID 1002 (Mechanical Keyboard) ditambahkan. Harga: Rp3500000.0

=====
                        DETAIL TRANSAKSI ORDER
=====
Nomor Order      : ORD-501
Tanggal Order    : 2026-09-22
ID Pelanggan     : 101
Nama Pelanggan   : Agam Darmawan
Alamat           : Pakisjajar Permai Blok D No 9, Malang
Nomor Telepon    : 08123456789
-----
Daftar Barang yang Dipesan:
  1. Produk ID: 1001 [Laptop Gaming] - Rp8500000.0
  2. Produk ID: 1002 [Mechanical Keyboard] - Rp3500000.0
-----
TOTAL PEMBAYARAN: Rp8850000.0
=====
```

2.

```
=====
                        SISTEM MANAJEMEN SERVIS BENGKEL MAJU
=====

-----
Nama Pelanggan   : Agam Yoga Darmawan (No. Telp: 08123456789)
Kendaraan        : Toyota Toyota GR Yaris [Car]
Nomor Polisi     : N 1234 AB
Teknisi / Staf   : Bambang Sugiharto (Mekanik Mobil)
Layanan Servis   : Tune Up & Ganti Oli Mesin Mobil
Biaya Servis     : Rp 350.000,00
Biaya Tambahan   : Rp 50.000,00 (Car)
TOTAL ESTIMASI   : Rp 400.000,00
-----

Nama Pelanggan   : Tesalonika Grace Benila (No. Telp: 08345678901)
Kendaraan        : Honda Mitsubishi Lancer Evo X [Car]
Nomor Polisi     : G 8888 HON
Teknisi / Staf   : Bambang Sugiharto (Mekanik Mobil)
Layanan Servis   : Servis Sistem Pengereman Mobil
Biaya Servis     : Rp 200.000,00
Biaya Tambahan   : Rp 50.000,00 (Car)
TOTAL ESTIMASI   : Rp 250.000,00
-----

Nama Pelanggan   : Agam Yoga Darmawan (No. Telp: 08123456789)
Kendaraan        : Yamaha RX-King [Motorcycle]
Nomor Polisi     : N 5678 CD
Teknisi / Staf   : Dedi Irawan (Mekanik Motor)
Layanan Servis   : Servis Ringan & Ganti Oli Motor
Biaya Servis     : Rp 95.000,00
Biaya Tambahan   : Rp 20.000,00 (Motorcycle)
TOTAL ESTIMASI   : Rp 115.000,00
-----

-----
Nama Pelanggan   : Tesalonika Grace Benila (No. Telp: 08345678901)
Kendaraan        : Honda Vario 150 [Motorcycle]
Nomor Polisi     : G 0831 EF
Teknisi / Staf   : Dedi Irawan (Mekanik Motor)
Layanan Servis   : Servis Lengkap & Pembersihan CVT
Biaya Servis     : Rp 150.000,00
Biaya Tambahan   : Rp 20.000,00 (Motorcycle)
TOTAL ESTIMASI   : Rp 170.000,00
-----

Semua data servis kendaraan berhasil diproses.
PS C:\code\PraktikumPBO>
```

---- Good Luck ----